> **Info**
>
> This document uses Obsidian markdown, it may not render properly on some viewers

False Position Method

Theoretical development

The **False Position (Regula Falsi) Method** relies on approximating the function by a **secant line** over an interval where a root is known to exist.

To apply the method, we require a continuous function on a closed interval $[a, b]$ such that the function changes sign:

$$f(x) \in C^0([a, b]),\ f(a)f(b) < 0 \tag{condition}$$

This guarantees, by **Bolzano's theorem**, the existence of at least one root $r \in (a, b)$.

Linear approximation on an interval

Instead of expanding around a single point (as in Newton), we approximate f using the **line passing through the endpoints**:

$$(a, f(a)),\ (b, f(b))$$

The equation of this secant line is:

$$s(x) = f(a) + \frac{f(b) - f(a)}{b - a}(x - a)$$

Solving this for the root at 0:

$$c = a - \frac{f(a)(b - a)}{f(b) - f(a)} \tag{recurrence}$$

The recurrence uses the following facts:

- If $f(a)f(c) < 0$, then the root lies in (a, c), so set $b := c$
- Otherwise, the root lies in (c, b), so set $a := c$

Note

The idea of the algorithm now becomes:
Construct a linear approximation of fff over an interval containing a root, compute the zero of this approximation, and iteratively shrink the interval by preserving the sign change.

The process is repeated until the approximation is within a desired tolerance, the interval becomes sufficiently small, or the maximum number of iterations is reached.

Building the algorithm

Now we actually build it. We'll use C++ 26 as it's the latest standard.
Let's start by defining our includes and constants:

```
#include <cmath>
#include <print>
#include <iostream>

// Tolerance
constexpr double EPS = 1e-6;
// Max iterations
constexpr size_t MAX_ITER = 256;
```

Here the constant "EPS" is our precision (10^{-6}) and "MAX_ITER" is the maximum amount of iteration we can run. Since the method converges rather quickly, this is ample time.
We now define some templates and types, this is standard generic programming stuff, as we want the method to accept any function.

```
// Real functions template
template<typename F>
concept RealFunction =
        requires(F f, double x)
        {
            { f(x) } -> std::convertible_to<double>;
        };
```

```
// Function type signature
using Func = std::function<double(double)>;
```

We want the following driver code:

```
// Functions to test
std::array<Func, 3> functions = {
    [](const double x) { return x * x - std::cos(x); },
    [](const double x) { return x * x * x - 1; },
    [](const double x) { return std::exp(x) - 10 * x; },
};

std::println("Testing false position method for functions:");
std::println("f(x) = x^2 - cos(x)");
std::println("f(x) = x^3 - 1");
std::println("f(x) = e^x - 10x\n");

// Testing
std::ranges::for_each(functions, [](auto &f) {
    double a{}, b{};
    std::println("Give an initial range [a, b] (space separated)");
    std::cin >> a >> b;

    try {
        double r{false_position(a, b, f)};
        std::println("False position method returned {}\n", r);
    } catch (const std::domain_error &e) {
        std::print("Method failed because: {}\n\n", e.what());
    } catch (const std::runtime_error &e) {
        std::print("{}\n\n", e.what());
    }});

return 0;
```

This is good, scalable, code. We have a data structure representing our inputs, and a range-based for loop iterating it, properly handling errors on the method.

Now for the actual implementation:

```
// The method
template<RealFunction F>
auto false_position(const double a, const double b, const F &f) -> double {
    double la{a}, lb{b}, c{}, fa{f(la)}, fb{f(lb)}, fc{};
    if (fa * fb >= 0) throw std::domain_error("Bolzano's theorem not
satisfied");
```

```
    size_t i{1};
    while (i <= MAX_ITER) {
        if (std::abs(fb - fa) < EPS)
            throw std::runtime_error("Division by zero: denominator too
small");

        c = la - ((fa * (lb - la)) / (fb - fa));
        fc = f(c);

        std::println("Iteration {}: a = {}, b = {}, c = {}", i, la, lb, c);

        if (std::abs(fc) < EPS or std::abs(lb - la) < EPS) return c;

        if (fc * fa < 0) {
            lb = c;
            fb = fc;
        } else {
            la = c;
            fa = fc;
        }

        ++i;
    }
    // No convergence
    throw std::runtime_error("Max iteration limit exceeded");
}
```

The function takes the initial (a, b) interval and the function. We first check that Bolzano's Theorem holds.
Then, we start iterating running with the recurrence relation, only breaking out if we get a division by zero or exceed the iteration limit.

Analysis of the algorithm and conclusion

Here's some test data:

$$f_1(x) = x^2 - cos(x)$$
$$f_2(x) = x^3 - 1$$
$$f_3(x) = e^x - 10x$$

Test for f_1

Expected value: 0.82413231230252242296

Initial range (a, b) *:* 0, 2

Iteration 1: a = 0, b = 2

Iteration 2: a = 0.3692662072055313, b = 2, c = 0.6183744350439322

Iteration 3: a = 0.6183744350439322, b = 2, c = 0.7415987525153498

Iteration 4: a = 0.7415987525153498, b = 2, c = 0.7928309035767985

Iteration 5: a = 0.7928309035767985, b = 2, c = 0.812527532827263

Iteration 6: a = 0.812527532827263, b = 2, c = 0.8198669311304935

Iteration 7: a = 0.8198669311304935, b = 2, c = 0.8225695744776313

Iteration 8: a = 0.8225695744776313, b = 2, c = 0.8235604357376303

Iteration 9: a = 0.8235604357376303, b = 2, c = 0.8239231271650416

Iteration 10: a = 0.8239231271650416, b = 2, c = 0.8240558071516201

Iteration 11: a = 0.8240558071516201, b = 2, c = 0.8241043337394038

Iteration 12: a = 0.8241043337394038, b = 2, c = 0.8241220805282534

Iteration 13: a = 0.8241220805282534, b = 2, c = 0.8241285705665967

Iteration 14: a = 0.8241285705665967, b = 2, c = 0.8241309439623149

Iteration 15: a = 0.8241309439623149, b = 2, c = 0.8241318119056416

Iteration 16: a = 0.8241318119056416, b = 2, c = 0.8241321293093294

False position method returned 0.8241321293093294

Test for f_2

Expected value: 1

Initial range (a, b) *:* 0, 2

Iteration 1: a = 0, b = 2, c = 0.25

Iteration 2: a = 0.25, b = 2, c = 0.4657534246575342

Iteration 3: a = 0.4657534246575342, b = 2, c = 0.6403630266802741

Iteration 4: a = 0.6403630266802741, b = 2, c = 0.7699424960309564

Iteration 5: a = 0.7699424960309564, b = 2, c = 0.8585771251953758

Iteration 6: a = 0.8585771251953758, b = 2, c = 0.9154532149617496

Iteration 7: a = 0.9154532149617496, b = 2, c = 0.9503612361259737

Iteration 8: a = 0.9503612361259737, b = 2, c = 0.9711796483435732

Iteration 9: a = 0.9711796483435732, b = 2, c = 0.9833781169037836

Iteration 10: a = 0.9833781169037836, b = 2, c = 0.9904509283841171

Iteration 11: a = 0.9904509283841171, b = 2, c = 0.9945266192248776

Iteration 12: a = 0.9945266192248776, b = 2, c = 0.9968668475591339

Iteration 13: a = 0.9968668475591339, b = 2, c = 0.9982078234034287

Iteration 14: a = 0.9982078234034287, b = 2, c = 0.9989753090132928

Iteration 15: a = 0.9989753090132928, b = 2, c = 0.9994142694126268

Iteration 16: a = 0.9994142694126268, b = 2, c = 0.999665233787777
Iteration 17: a = 0.999665233787777, b = 2, c = 0.9998086844367171
Iteration 18: a = 0.9998086844367171, b = 2, c = 0.9998906700980669
Iteration 19: a = 0.9998906700980669, b = 2, c = 0.9999375235748394
Iteration 20: a = 0.9999375235748394, b = 2, c = 0.9999642984686835
Iteration 21: a = 0.9999642984686835, b = 2, c = 0.9999795988908509
Iteration 22: a = 0.9999795988908509, b = 2, c = 0.9999883421468972
Iteration 23: a = 0.9999883421468972, b = 2, c = 0.9999933383446933
Iteration 24: a = 0.9999933383446933, b = 2, c = 0.9999961933316738
Iteration 25: a = 0.9999961933316738, b = 2, c = 0.9999978247582949
Iteration 26: a = 0.9999978247582949, b = 2, c = 0.9999987570038709
Iteration 27: a = 0.9999987570038709, b = 2, c = 0.9999992897162139
Iteration 28: a = 0.9999992897162139, b = 2, c = 0.9999995941234582
Iteration 29: a = 0.9999995941234582, b = 2, c = 0.9999997680705173
False position method returned 0.9999997680705173

Test for f_3

Expected value: 0.11183255915896296483
Initial range (a, b) : 0, 0.5
Iteration 1: a = 0, b = 0.5, c = 0.11490875007229216
Iteration 2: a = 0, b = 0.11490875007229216, c = 0.11185331684507092
Iteration 3: a = 0, b = 0.11185331684507092, c = 0.11183269905905817
Iteration 4: a = 0, b = 0.11183269905905817, c = 0.11183256010183668
False position method returned 0.11183256010183668

Analysis

Notice that the method converges rather slowly (compared to Newton's at least), hence it's not preferred over others. It's benefit over Newton's however, is that it does not require the derivative of the function.

Pitfalls

1. False Position may not converge, this happens when there is more than 1 root in the interval, checking for that requires derivatives.
2. Can stall for really small values of EPS

Conclusion

Obviously to do a real benchmark that would require many more functions, margins of error and initial guesses. However, that seams to be beyond the scope of this project.
Overall, the result were pretty satisfactory, as we achieved all proposed goals.